

Highlights

Terrain Fine-Detail Blending in Lunar-Landing Simulations: Comparing Smoothstep, Gaussian, and Hybrid Blending Methods in Isaac Lab

Haktan Yusuf Güloğlu

- As-shipped Gaussian terrain blending is $\sim 3200\times$ worse in C^0 continuity than smoothstep.
- A calibrated Gaussian and a new hybrid blend restore near machine-precision continuity.
- Single-seed Isaac Sim training seemed to favor Gaussian and hybrid over smoothstep.
- Extending to three seeds finds no significant difference among blends (all $p \geq 0.20$).
- A concrete case study on how misleading single-seed RL comparisons can be in practice.

Terrain Fine-Detail Blending in Lunar-Landing Simulations: Comparing Smoothstep, Gaussian, and Hybrid Blending Methods in Isaac Lab

Haktan Yusuf Güloğlu^{a,*}

^a*Sakarya University, National Technology Workshop, Artificial Intelligence Center, Sakarya, Türkiye*

ARTICLE INFO

Keywords:

Lunar landing simulation
Reinforcement learning (Soft Actor-Critic)
Isaac Lab
Procedural terrain generation
Multi-resolution blending
Continuity analysis

ABSTRACT

In procedurally generated multi-resolution terrain, the shape of the blending function joining a coarse global surface to a fine-detail local patch affects the quality of physically queryable contact geometry, yet this choice is usually a visual concern, not evaluated quantitatively for physics fidelity. This study compares the compact-support smoothstep blend used in a SAC-controlled, gimbal-thrust lunar lander in Isaac Lab against the infinite-support Gaussian blend inherited from a legacy codebase, under an equal transition-radius budget, and proposes a hybrid method using Gaussian blending near the landing-gear contact footprint and smoothstep beyond it. Across 30 seeds, analytic measurements show the as-shipped Gaussian produces roughly 3200 times worse boundary height continuity (C0, RMS) and 530 times worse slope continuity (C1) than smoothstep (Wilcoxon signed-rank, $p < 1 \times 10^{-8}$), a scale-invariant defect; a recalibrated Gaussian and the hybrid both restore near-machine-precision continuity, with negligible VRAM impact. Isaac Sim training runs (5.73 million steps per method) were first conducted with a single seed, where the calibrated Gaussian and hybrid appeared to reach a markedly higher final success rate than smoothstep. Extending the comparison to two more seeds ($n = 3$ total) found no statistically significant difference between methods on any metric (all Mann–Whitney $p \geq 0.20$); the apparent gap is largely consistent with ordinary seed variance. This gives a concrete demonstration of how misleading single-seed RL comparisons can be: the blend function has a robust, measurable effect on boundary continuity, but this does not translate into a distinguishable RL training-performance difference at the present scale and $n = 3$.

1. Introduction

Autonomous lunar landing is a control problem with direct operational consequences: during the powered-descent phase it requires real-time precision positioning and safe touchdown over a cratered, rocky, and sloped terrain. In recent years, GPU-parallel physics engines (NVIDIA PhysX) and the simulation frameworks built on them have made it practical to learn complex, contact-rich tasks with deep reinforcement learning (RL) by running thousands of environments concurrently [1–3]. The project underlying this study belongs to this class: an Isaac Lab direct-workflow task in which a four-legged, dual-axis gimbal-thrust rocket learns to land, using Soft Actor-Critic (SAC), on a procedural lunar terrain that layers craters, rocks, and centimeter-scale regolith texture on top of a macro surface derived from a real digital elevation model (DEM).

In procedural multi-resolution terrain of this kind, two layers are typically used to keep the computation and memory budget bounded: a low-resolution global surface covering a wide area, and a high-resolution local patch generated only where needed, near the agent or the target. Seamlessly joining these two layers is a classic problem that has been studied in the computer-graphics literature for more than twenty-five years (geometry clipmaps: [4]; ROAM: [5]; geomipmapping: [6]). However, this literature focuses almost entirely on rendering quality; how a discontinuity at the seam affects the leg’s contact with the ground, altimeter readings, or the noise character of an RL agent’s proprioceptive observation is rarely addressed quantitatively.

This problem manifests in the system under study as two different and mutually inconsistent solutions: the training environment blends the layers with a compact-support cubic smoothstep function, while a

*Corresponding author.

 haktanguloglu24@gmail.com (H.Y. Güloğlu)

previously used mesh-based implementation uses an infinite-support Gaussian bell. These two functions differ in a fundamentally different mathematical property: smoothstep decays to exactly zero at a finite radius (compact support), whereas a standard Gaussian bell never reaches exactly zero at any finite radius (infinite support; [7]) and must necessarily be truncated at some point to be used in an engineering application; this truncation risks creating a new, measurable seam at the cut point. Comparing the two methods fairly requires forcing both to zero at the same transition radius (hard truncation) and measuring them under the same transition budget; to the best of our literature search, this comparison has not been directly addressed in any terrain-randomization study (see Section 2).

Motivated by these observations, this study asks three research questions:

- **RQ1:** When compact-support (smoothstep) and infinite-support (Gaussian) blending functions are compared fairly under the same transition-radius budget, how large is the quantitative difference in boundary continuity ($C0/C1$), and what trade-off does this form with computation/memory cost?
- **RQ2:** Can a hybrid blending method that uses Gaussian blending in the landing-gear contact region and smoothstep beyond it combine the continuity advantages of both methods at a reasonable increase in computational cost?
- **RQ3:** Do the continuity differences among these three blending methods translate into a measurable learning-performance difference in a real Isaac Sim/Isaac Lab SAC training run?

Accordingly, the aim of this study is threefold: (i) to quantitatively compare the existing smoothstep method and the previously used Gaussian method within the same architecture and under the same transition budget; (ii) to propose and validate a hybrid method that combines the advantages of both; (iii) to compare these three methods not only at the analytic/CPU level but inside a real Isaac Sim/Isaac Lab GPU-parallel physics simulation with a real SAC training run, to show whether the choice of blending function is reflected in learning performance. This last point is central to the study’s original contribution: the comparison is carried out not as an abstract numerical experiment but on real lunar terrain, inside a real Isaac Sim training loop.

The main contributions of this study are as follows:

- The existing smoothstep method and the previously used Gaussian blending method are, for the first time, fairly compared under the same transition budget by truncating both to zero at a finite transition radius (hard truncation); it is shown that the untruncated Gaussian, which appears to behave well, in fact carries a hidden seam defect that emerges once it is truncated at a finite radius.
- A new hybrid blending method is proposed that uses a Gaussian weighted toward the contact region and a smoothstep weighted beyond it, and this method is shown to provide machine-precision continuity without creating a new seam at its own internal transition.
- A statistically validated (Wilcoxon signed-rank test) comparison of continuity, cost, and VRAM is carried out over thirty seeds and four method variants (smoothstep, as-shipped Gaussian, calibrated Gaussian, hybrid); the robustness of the findings is tested with four separate ablation analyses (contact radius, switch width, transition radius, grid resolution).
- The three methods are compared inside a real Isaac Lab/Isaac Sim PhysX environment with GPU-parallel SAC training; unlike prior work in the terrain-randomization literature, this is, to our knowledge, the first attempt to treat the choice of blending function directly as a training-design variable.
- This real-training comparison is carried out not with a single seed per method but with three independent seeds ($n = 3$); it is shown that the large performance gap apparent in the initial single-seed measurement could not be statistically confirmed with multiple seeds, providing a concrete, self-contained cautionary example about the reliability of single-run RL comparisons.
- All three methods are integrated into the production code in a backward-compatible way that does not change existing behavior, and are selectable via an environment variable (`ISAACLAB_TERRAIN_BLEND_METHOD`).

The remainder of this paper is organized as follows. Section 2 reviews the literature in which this study is positioned, under five thematic clusters. Section 3 details the experimental environment and the simulation setup shared by all conditions, the mathematical definition of the blending functions, the analytic evaluation protocol, and the real Isaac Sim training protocol. Section 4 presents the continuity, cost, VRAM, ablation, and real-training results. Section 5 discusses the findings in relation to the literature and addresses limitations. Section 6 summarizes the study and concludes with concrete directions for future work.

2. Related Work

This section reviews the literature in which this study is positioned, under five thematic clusters. The search is a targeted/scoping review, not a PRISMA-compliant systematic review, and within each cluster the most-cited/foundational references of the field are prioritized.

Cluster 1 addresses multi-resolution blending and level-of-detail (LOD) transition in procedural terrain. Geometry clipmaps [4], ROAM [5], and geomipmapping [6] form exactly the traditional computer-graphics line to which this study’s two-mesh (global + local patch) framework belongs; all three treat the crack/seam that forms at the boundary between low- and high-resolution meshes as a central engineering problem. This study’s finding — that the as-shipped Gaussian leaves a real seam once truncated at a finite radius — is a concrete, measured instance of exactly the same class of problem this literature has been solving for more than twenty-five years; smoothstep’s compact support, in turn, coincides with the principle, used by ROAM and geoclipmaps, of a weighting function that reaches exactly zero at the boundary.

Cluster 2 addresses the continuity properties of compact-support and infinite-support blending functions. The multiquadric/Gaussian-type radial basis function that Hardy [7] proposed for topography belongs, like the as-shipped Gaussian in this study, to an infinite-support family that never reaches exactly zero at any finite radius; this coincides with the measured residual behavior (approximately $\exp(-1) \approx 37\%$). Wendland [8] formalizes the solution to exactly this problem: compact-support, minimal-degree positive-definite radial functions for a given degree of smoothness; smoothstep (cubic Hermite, [9]) is a one-dimensional, low-degree special case of this family. Ohtake et al. [10] describe the strategy that this study’s hybrid method follows: building a multi-scale hierarchy from compact-support basis functions to obtain different blending behavior at different scales.

Cluster 3 addresses terrain modeling and contact dynamics in planetary/lunar landing simulations. The goal of the ALHAT program [11, 12] — precision landing and LiDAR-based terrain-relative navigation within a hundred meters — and the accuracy level offered by LOLA/SELENE-based digital elevation models [13] show that terrain accuracy in real lunar-landing engineering is not a cosmetic concern but a direct operational requirement. This reinforces the study’s “why the seam/continuity matters” motivation: a discontinuity in the foot-contact region corresponds to a class of error that could affect hazard detection and landing-site-selection decisions in real ALHAT-class systems. This motivation is also corroborated by recent flight data: the vision-based navigation and obstacle-detection results of JAXA’s SLIM mission (January 2024) [14] showed that real-time obstacle/terrain detection performed 50 m before touchdown provided meter-scale landing precision; this makes concrete that errors in terrain representation translate directly into landing safety not only in simulation but in real flight.

Cluster 4 addresses procedural terrain randomization in physics-based RL environments. Rudin et al. [3] use, on IsaacGym, a difficulty-based terrain curriculum similar to this study’s terrain pool, showing that this study’s curriculum and terrain-pool design is not an isolated instance but an established practice in the field. Mittal et al. [1] (Orbit) and NVIDIA [2] (Isaac Lab) define the architectural foundation of the platform this study uses directly, while Tobin et al. [15] provide the general motivation for procedural randomization in sim-to-real transfer. However, none of these sources treats the multi-resolution blending function itself (smoothstep, Gaussian, or hybrid) within terrain randomization as a design variable; all focus on terrain diversity, not on how the shape of the blend affects continuity and learning cost. This study is positioned exactly in that gap.

Cluster 5 addresses hybrid/regional blending-function composition (partition of unity). Ohtake et al. [10] are again the central source here: the multi-scale, partition-of-unity-style composition of compact-support basis functions is the general mathematical framework for this study’s hybrid method (one function near the contact region, another beyond it, with a smooth switch between them). Under this framework, it is an

Overall methodology pipeline

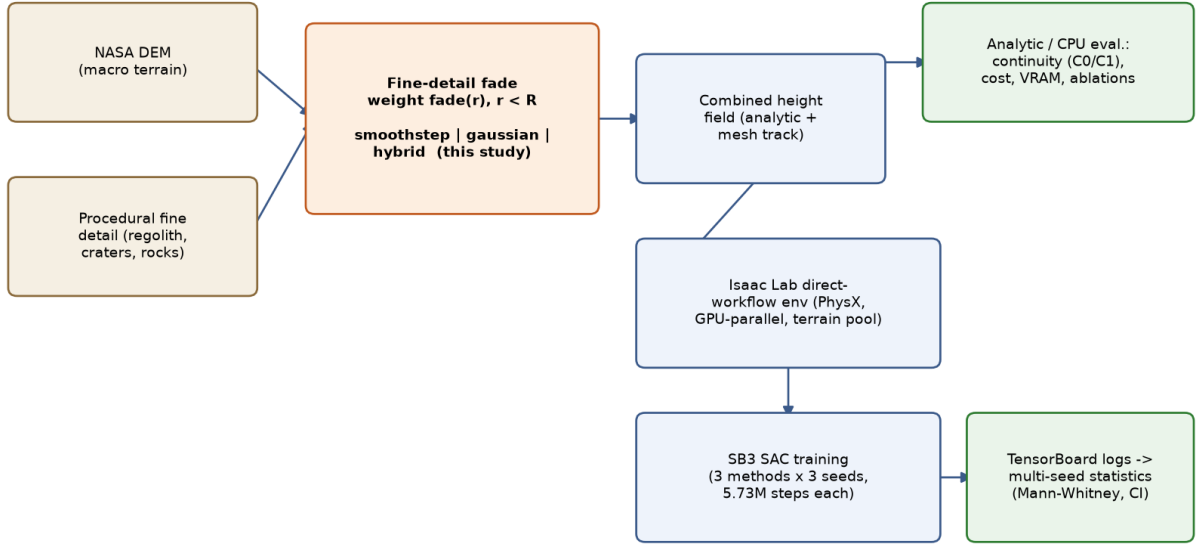


Figure 1: Overall methodology pipeline: from terrain generation (DEM + procedural fine detail), through the choice of blending function, to analytic/CPU evaluation and the real Isaac Lab/SAC training-analysis pipeline.

expected result, consistent with the literature, that the hybrid switch does not create a new seam at machine precision: a convex combination of two C1-smooth functions with a C1-smooth weight is itself C1-smooth.

In summary, Cluster 1 supplies the problem class (seam/continuity at a multi-resolution transition) while that literature focuses mainly on visual quality; this study measures the same problem numerically on a physically queryable height field (foot contact). Cluster 2 grounds why the empirical finding is true in a mathematical result more than fifty years old; this is not a coding bug but a known structural property of radial basis functions. Cluster 3 supplies the real-world engineering justification for “why it matters.” Cluster 4 clarifies this study’s place in the literature and the gap it fills. Cluster 5 places the proposed hybrid method within an established mathematical framework rather than leaving it as an isolated engineering trick.

3. Methods

This section consists of five subsections: the experimental environment and task definition, the simulation setup shared by all conditions, the mathematical formulation of the terrain representation and blending problem, the analytic (CPU) evaluation protocol, and the real Isaac Lab/Isaac Sim training protocol. All three blending methods are evaluated first at the analytic/CPU level and then inside the real GPU-parallel physics simulation, under the same fixed transition radius and the same hard-truncation rule; this consistency ensures that the findings from the two evaluation layers are comparable.

3.1. Experimental Environment and Task Definition

The experimental environment is **LunarRocket-Lander-Direct-v0**, a task defined on the Isaac Lab direct-workflow API. The agent represents a four-legged, dual-axis gimbal-thrust rocket; the observation space includes position, velocity, orientation, angular velocity, IMU and altimeter readings, and position relative to the target; the action space covers the main thrust magnitude and the gimbal angle. The agent is trained with the Stable-Baselines3 (SB3) implementation of Soft Actor-Critic (SAC), concurrently, on hundreds of GPU-parallel environment instances running on NVIDIA PhysX.

The terrain is generated not in a single procedural step but through a two-layer representation: (i) a global macro surface derived from a real NASA digital elevation model (DEM), enriched with randomized slope, craters, and rock fields; (ii) a high-resolution local regolith pattern generated near the agent’s start and target positions using a centimeter-scale, multi-frequency noise function. These two layers are loaded onto the GPU by copying a slot from a pre-built “terrain pool” prepared by a background worker-thread pool during environment reset; active episodes are never changed mid-episode, and new generations only take effect at reset. Training proceeds under a curriculum in which the spawn/target distance and the commandable gimbal authority are increased gradually, gated by a rolling-window success rate.

Where these two layers meet, the fine detail of the local patch is added to the global surface by “fading” it out within a fixed radius; the mathematical shape of this fade function is the subject of this study and is defined in Section 3.3.

Fig. 2 and Fig. 3 are images captured directly from the simulation while running a trained policy in the real Isaac Lab/Isaac Sim environment (offscreen render, via a checkpoint loaded with `ISAACLAB_CHECKPOINT`; captured during the final phase of the landing approach, at approximately 0.6–0.7 m above the ground). Two changes were made for this capture: (i) the scene’s default shadowless dome light was dimmed and a low-angle directional “sun” light that casts real shadows was added, making the terrain relief and shadows visible; (ii) the landing region was rendered with the real, high-detail 512×512 local patch (see Section 3.1, “local regolith pattern”) in addition to the 128×128 global surface, so that the centimeter-scale fine texture (dust/grain-level roughness) is also visible. The claim that the vehicle is upright rests not on assumption but on the simulation’s own body-orientation data: throughout this run the alignment of the body’s local up axis with the world $+z$ axis (up_z) measured a constant $+1,000$, i.e., the vehicle is not tipped over but exactly vertical. Fig. 2 shows the four-legged lander over a real cratered/rocky lunar surface; the ridge line, grainy/rough surface texture, shadows, and scattered rocks are the working result of the DEM-based macro surface and the procedural rock/crater/fine-detail fields defined in Section 3.1. Fig. 3 is a close-up of the same lander in the same frame; the body, the tips of the dual-axis gimbal thrust axis, and the grainy texture of the ground are visible together. These images are presented solely to document the environment visually; they are not claimed to represent any particular landing outcome (success/failure).

3.2. Simulation Setup

This subsection collects, in one place, the simulation configuration shared by both the analytic (CPU) protocol and the real Isaac Lab/Isaac Sim training runs (Table 1). Physics is solved GPU-parallel on NVIDIA PhysX with a fixed step of $1/120$ second (120 Hz); the policy produces actions at 60 Hz with `decimation = 2`, and each episode lasts 20 s, i.e., 1200 control steps. Lunar gravity (1.62 m s^{-2}), a low-restitution regolith-like contact material, and 512 concurrent environment instances are held fixed across all conditions.

Vehicle and actuation parameters (mass, thrust envelope, physical and policy gimbal limits, TVC actuator gains) and the SAC hyperparameters are not changed across the three blending methods. The only quantity that varies from method to method is the fine-detail fade function defined in Section 3.3 and its associated parameters (the last group of Table 1; selected via the `ISAACLAB_TERRAIN_BLEND_METHOD` environment variable). This fixing is required so that the differences observed in Section 4 can be attributed solely to the blending method.

Owing to the computational budget constraint, the real training comparison was carried out with a shortened version of the full-scale (1200-iteration) configuration — three independent seeds per method, approximately 5.73 million environment steps per run; this scale limitation is addressed further in Section 3.5 and Section 4.3.

3.3. Terrain Representation and Blending Functions

The fine-detail height of the local patch is added to the global surface via one of three candidate fade functions. All three are forced to exactly zero for $r \geq R$ (hard truncation), where r is the distance from the center and $R = \text{terrain_detail_radius_m} = 8.0$ m is the fixed transition radius; this is the core methodological arrangement that makes the three methods comparable under the same transition budget, and it is implemented this way in the production code as well.

Smoothstep (compact support, default/current method): with $t = \text{clamp}(1 - r/R, 0, 1)$,

$$\text{fade}(r) = t^2 (3 - 2t). \quad (1)$$

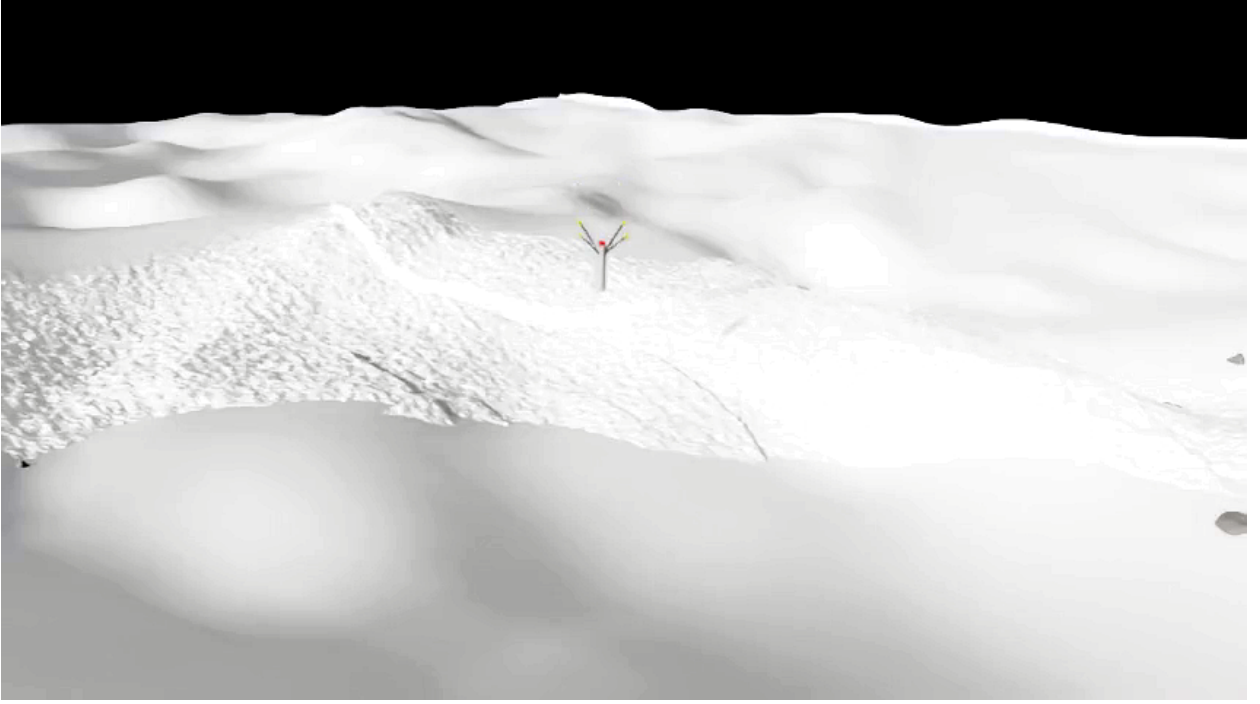


Figure 2: Scene captured from the trained policy running in the real Isaac Sim environment: the four-legged lander over a cratered/rocky, grain-textured lunar surface (high-detail local patch + directional light added for documentation purposes; the vehicle is confirmed exactly vertical by simulation data, $up_z = +1,000$).

This function is already exactly zero at $r = R$; it needs no additional truncation step, and C1 continuity is analytically guaranteed.

Gaussian (infinite support, previously used method):

$$\text{fade}(r) = \exp(-k (r/R)^2). \quad (2)$$

Two values of k are evaluated separately in this study: the previously used (as-shipped) constant $k = 1.0$, and the calibrated constant proposed in this study, $k = \ln(1000) \approx 6.9078$, chosen so that the practical support of the Gaussian decays to about one part in a thousand at R . The `cfg.terrain_gaussian_k` field in the production code uses the calibrated constant by default; the as-shipped $k = 1.0$ is tested only in this study's analytic/CPU comparison, as a separate variant (`gaussian_shipped`), to demonstrate the existing defect. For both values of k , the function is forced to zero from the outside for $r \geq R$.

Hybrid (proposed method): with landing-gear contact-footprint radius $r_c = \text{terrain_hybrid_contact_radius_m} = 1.5$ m and switch width $w = \text{terrain_hybrid_switch_width_m} = 0.3$ m, a smoothstep-weighted switch(r) is defined over $[r_c - w/2, r_c + w/2]$:

$$u = \text{clamp}\left(\frac{hi - r}{hi - lo}, 0, 1\right), \quad \text{switch}(r) = u^2 (3 - 2u), \quad (3)$$

where $lo = r_c - w/2$, $hi = r_c + w/2$. The final fade is defined as

$$\text{fade}(r) = \text{switch}(r) \cdot \text{gauss}(r) + (1 - \text{switch}(r)) \cdot \text{smoothstep}(r) \quad (4)$$

i.e., the Gaussian is weighted near the landing-gear contact region and smoothstep is weighted beyond it, with a smoothstep-shaped, itself C1-smooth weight used to smooth the transition between the two functions. Since $\text{switch}(r) = 0$ at the boundary $r = R$, the hybrid method is analytically identical to smoothstep at that boundary.



Figure 3: A close-up view of the lander in the same scene: the body, the tips of the gimbal thrust axis, and the centimeter-scale grainy texture of the ground are visible together.

These three methods are integrated into the production code, selectable via the `ISAACLAB_TERRAIN_BLEND_METHOD` environment variable (`smoothstep` | `gaussian` | `hybrid`), without changing the existing default behavior (`smoothstep`).

3.4. Analytic (CPU) Evaluation Protocol

The analytic evaluation was carried out, without any Isaac Lab/torch dependency, on a one-to-one NumPy port of the formulas in the production code, using two independent tracks: (i) mesh-track, a track that generates an actual triangle mesh and measures vertex count and time; (ii) analytic-track, a track that queries the height field directly as a continuous function. Cross-validating the two tracks was used to show that the findings are not an artifact specific to mesh discretization.

Continuity metrics were computed with finite differences: C0 (height) continuity is defined as the absolute value of the height difference between points immediately on either side of the transition boundary (spatial epsilon = 0.05 m); C1 (slope) continuity is defined as the angular difference of the local gradient at the same points (gradient epsilon = 0.02 m). Measurements were taken over 720 angular samples, and both maximum and RMS (root-mean-square) values are reported. In initial measurements, it was observed that measuring over the combined (global + local) height mixed with the epsilon-scale variation of the natural terrain slope and masked the actual seam signal; a “detail-only” measurement mode was therefore defined that isolates only the faded fine-detail component ($\text{fade}(r) \cdot \text{fine_detail}(x, y)$), and this was adopted as the study’s headline metric; the measurement over the combined surface was retained as a secondary/realistic reference.

Memory cost was estimated separately for the two-mesh (literal, separate global and local meshes) and single-grid (baked, one combined mesh) accounting schemes, assuming 32 bytes per vertex and a 2x multiplier for the collision mesh. Query throughput was additionally measured, in microseconds, for the pure fade formula alone (fade-only, isolated from terrain sampling), assuming 512 environments and 64 global + 24 local + 4 fine-detail queries per environment (92 queries/environment total).

The main comparison was repeated over thirty independent random seeds for four method variants (`smoothstep`, as-shipped Gaussian, calibrated Gaussian, `hybrid`); statistical significance was assessed with the Wilcoxon signed-rank test on the $\log_{10}$ ratio differences computed per seed (this test was preferred over parametric tests because, in some comparisons — e.g., `hybrid` vs. `smoothstep`, which are analytically identical at $r = R$ — the difference distribution converges to zero variance, and parametric effect-size measures such as Cohen’s d degenerate).

Table 1

Simulation, vehicle, RL training, and terrain-blending parameters shared by the analytic protocol and the real Isaac Lab/Isaac Sim training runs. Only the last group (blending parameters) varies from method to method.

Group	Parameter	Value
Physics / simulation	Physics engine	NVIDIA PhysX 5 (GPU pipeline)
	Simulation step (dt)	1/120 s (120 Hz)
	Control frequency (decimation = 2)	60 Hz
	Episode length	20 s (1200 control steps)
	Gravity	0, 0, -1.62 m/s^2
	Number of parallel environments	512
	Environment spacing (env_spacing)	90 m
	Physics cloning	replicate_physics + clone_in_fabric (on)
	Friction (static / dynamic)	0.85 / 0.65 (combine: multiply)
Vehicle and actuation	Restitution	0.02
	Vehicle mass	1.66 kg (1.63 body + gimbal linkages)
	Maximum thrust	30 N
	Maximum commanded throttle	0.35 ($\approx 10.5 \text{ N}$ effective)
	Physical gimbal limit	$\pm 25^\circ$ (mechanical hard stop)
	Policy gimbal envelope	4° – 8° (scales with curriculum difficulty)
	Thrust lever arm	0.25 m
	TVC actuator (implicit)	effort 2.5, stiffness 10, damping 0.1, armature 0.002
	Observation / action dimension	125 / 3
RL training (SB3 SAC)	Initial altitude	25 m (upright orientation)
	Policy network	MLP [384, 256, 128], ELU
	Replay buffer	1 000 000
	learning_starts	16 384
	Batch size	1024
	Learning rate	3×10^{-4}
	γ / τ	0.99 / 0.005
	train_freq / gradient_steps	1 / 32
	Entropy coefficient	automatic (target_entropy = -1.5)
Blending parameters	Normalization	normalize_input + normalize_value on, clip_obs = 10
	Comparison budget	350 iter $\times$ 32 $\times$ 512 $\approx 5.73 \text{ M}$ env. steps
	Seeds	42, 7, 123 ($n = 3$ per method, consecutive)
	Device	single GPU (cuda:0)
	Transition radius (R)	8.0 m
	Method options	smoothstep / gaussian / hybrid (ISAACLAB_TERRAIN_BLEND_METHOD)
	Calibrated Gaussian coefficient (k)	$\ln(1000) \approx 6.9078$
	Hybrid contact radius (r_c)	1.5 m
	Hybrid switch width (w)	0.3 m
	Fine-detail amplitude range	0.006–0.020 m
	Macro height / slope scale	0.45 m / 0.035
	Craters / rocks (per episode)	3 / 5
	Terrain pool (size / refresh)	15 slots / 60 assignments

The robustness of the findings was tested with four separate ablation analyses: (i) a contact-radius (r_c) sweep over 1.0–4.0 m; (ii) a switch-width (w) sweep over 0.05–2.0 m; (iii) a transition-radius (R) sweep over 4–16 m, to test the scale-invariance of the defect; (iv) a grid-resolution sweep over approximately 0.31–1.27 m grid spacing, estimating the empirical convergence order via log-log linear regression.

3.5. Real Isaac Lab/Isaac Sim Training Protocol

Following the analytic evaluation, the three methods (smoothstep, calibrated Gaussian, hybrid) were compared in a real Isaac Lab/Isaac Sim training environment. This is central to the study’s original contribution: the comparison was carried out not as an abstract numerical experiment but with a SAC training loop running inside a GPU-parallel PhysX physics simulation, on a real cratered/sloped lunar surface.

Each method, selected via the ISAACLAB_TERRAIN_BLEND_METHOD environment variable, was trained on the same hardware (single machine, single GPU), with the same hyperparameter configuration

Table 2

Boundary continuity ratios (relative to smoothstep), aggregated over thirty seeds, isolating only the fine-detail component.

Method	C0 ratio	C1 ratio	Wilcoxon p	n (seeds)
smoothstep (reference)	1.0	1.0	–	30
gaussian_shipped ($k=1.0$)	3192x	535x	1.86e-09	30
gaussian_calibrated ($k=\ln(1000)$)	9.3x	1.6x	1.86e-09	30
hybrid	1.0 (identical)	1.0 (identical)	n/a (analytically identical)	30

(`sb3_sac_cfg.yaml`), sequentially rather than concurrently, to avoid GPU/VRAM contention. Each run lasted 5,734,400 environment steps (approximately 350 iterations); owing to the computational budget constraint, this is a shortened but still meaningfully informative version of the full-scale (1200-iteration) configuration, and this is a scale limitation that must be stated explicitly.

The initial measurement was carried out with a single run per method, at the default seed in the SAC agent’s configuration file (`seed = 42`). Considering that this single-run design is exactly the kind of pitfall warned against by Henderson et al. [16], the study was extended with two additional independent seeds (`seed = 7` and `seed = 123`), yielding a total of three independent runs ($n = 3$) per method — the `ISAACLAB_SEED` environment variable was added to the production code for this purpose. Because of the GPU-parallel PhysX physics simulation’s own internal nondeterminism (thread scheduling, floating-point reduction order) and the timing-dependence of the terrain pool’s background CPU generation process, two runs with the same seed are not bit-identical repetitions of each other; the three seeds therefore sample not only agent-initialization variance but also this system-level nondeterminism.

Training metrics (success rate, timeout rate, harsh-landing rate, curriculum difficulty, entropy coefficient, xy-progress reward) were logged via TensorBoard and parsed from the event files with `tensorboard.backend.event_processing.EventAccumulator`. For each (method, seed) run, the final performance was reduced to a SINGLE summary value by averaging the logged points in that run’s last 10% window; cross-method comparisons were then carried out on these three independent summary values (as independent samples) using the Mann-Whitney U test and a 5000-resample bootstrap 95% confidence interval. This is the statistically correct unit, avoiding the pseudoreplication of treating autocorrelated (non-independent) points within a single run as fake-independent samples; however, statistical power is low with $n = 3$, and this is explicitly emphasized in Section 4.3 and Section 5.

4. Results

This section presents the boundary-continuity and computational-cost findings, then the results of the four ablation analyses, and finally the results of the real Isaac Sim training comparison. No method was repeated; data/design details are left to Section 3.

4.1. Boundary Continuity and Computational Cost

Table 2 summarizes, aggregated over thirty seeds, the headline continuity measurements that isolate only the faded fine-detail component, expressed as a ratio relative to smoothstep. The as-shipped Gaussian (`gaussian_shipped`, $k = 1.0$) produces, on average, 3192 times worse C0 (height) continuity and 535 times worse C1 (slope) continuity than smoothstep; both differences are highly significant by the Wilcoxon signed-rank test ($p = 1.86 \times 10^{-9}$, $n = 30$). The calibrated Gaussian ($k = \ln(1000)$) largely removes this defect, but still leaves a measurably higher residual error than smoothstep (approximately 9.3x in C0, 1.6x in C1). The hybrid method, being analytically identical to smoothstep at the boundary $r = R$ (`switch(R) = 0`), showed zero difference across all thirty seeds; this is not a measurement outcome but an expected consequence of the method’s definition.

Fig. 4 shows the combined (macro + fine-detail) height cross-section of the four method variants. At this scale, the four curves are almost perfectly superimposed, because the fine-detail amplitude (centimeter scale) is much smaller than the macro terrain scale (meter scale); the order-of-magnitude differences reported in Table 2 cannot be seen with the naked eye at this visual scale. This directly demonstrates why the “detail-only”

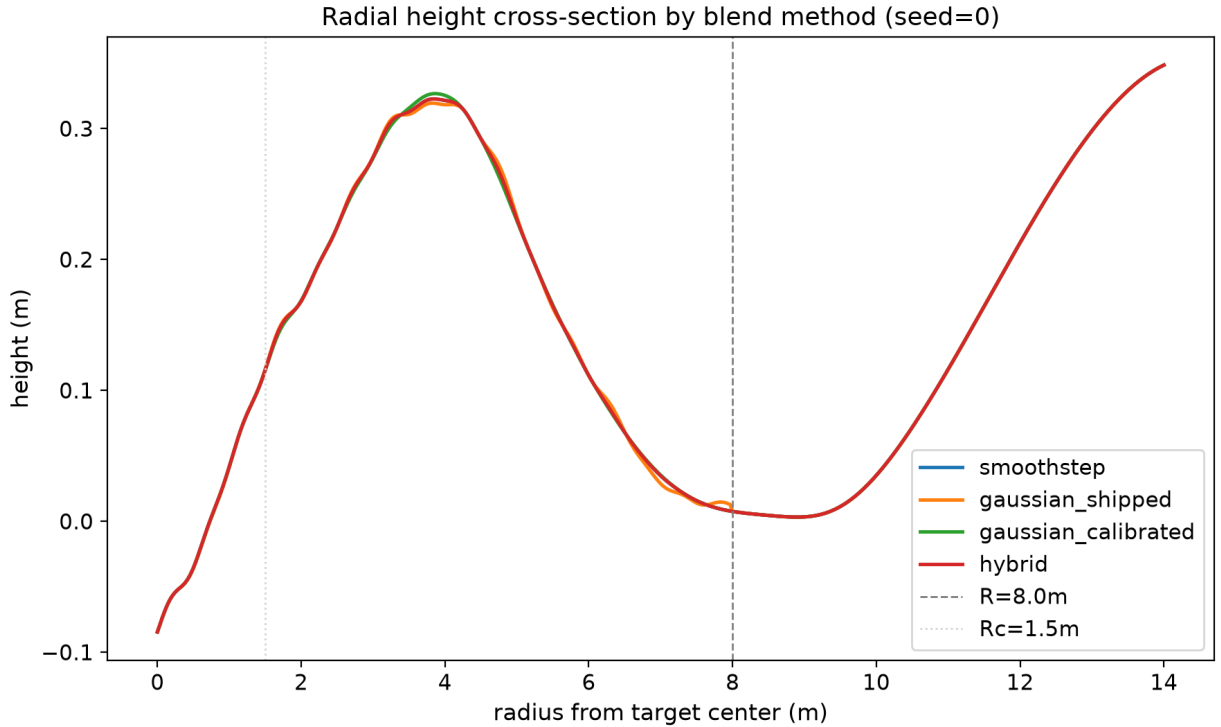


Figure 4: Radial fade profile of the three blending methods.

Table 3

Per-query computational cost of the fade formula.

Method	Cost ($\mu\text{s}/\text{query}$)
smoothstep	93.12
gaussian_shipped	148.84
gaussian_calibrated	149.61
hybrid	407.83

isolation step defined in Section 3.4 is necessary: the seam defect only appears once the fine-detail component is isolated.

Table 3 shows the computational cost, in microseconds, of the fade formula alone (isolated from terrain sampling). Smoothstep has the lowest cost (93.1 microseconds); the Gaussian variants (as-shipped and calibrated) have a similar, moderate cost (approximately 149–150 microseconds); the hybrid method has the highest cost (407.8 microseconds, approximately 4.3x that of smoothstep), because it evaluates both the Gaussian and smoothstep branches as well as the switch weight on every query.

Fig. 5 visualizes the analytic and mesh-track continuity measurements from Table 2 as per-method bar charts; **gaussian_shipped** having, in all four panels, an error bar orders of magnitude higher than the other three methods also visually confirms the magnitude difference reported in the table.

Table 4 shows the estimated memory for the two mesh-accounting schemes; this estimate depends only on mesh geometry (vertex count), and the shape of the blending method has no measurable effect on VRAM — the real VRAM lever is the mesh-topology choice (two-mesh vs. single-grid), not the choice of blending function.

Fig. 6 combines the cost metrics from Table 3 and Table 4 into a single view: mesh-track generation time and analytic-track query throughput are relatively close across methods, while the hybrid method's

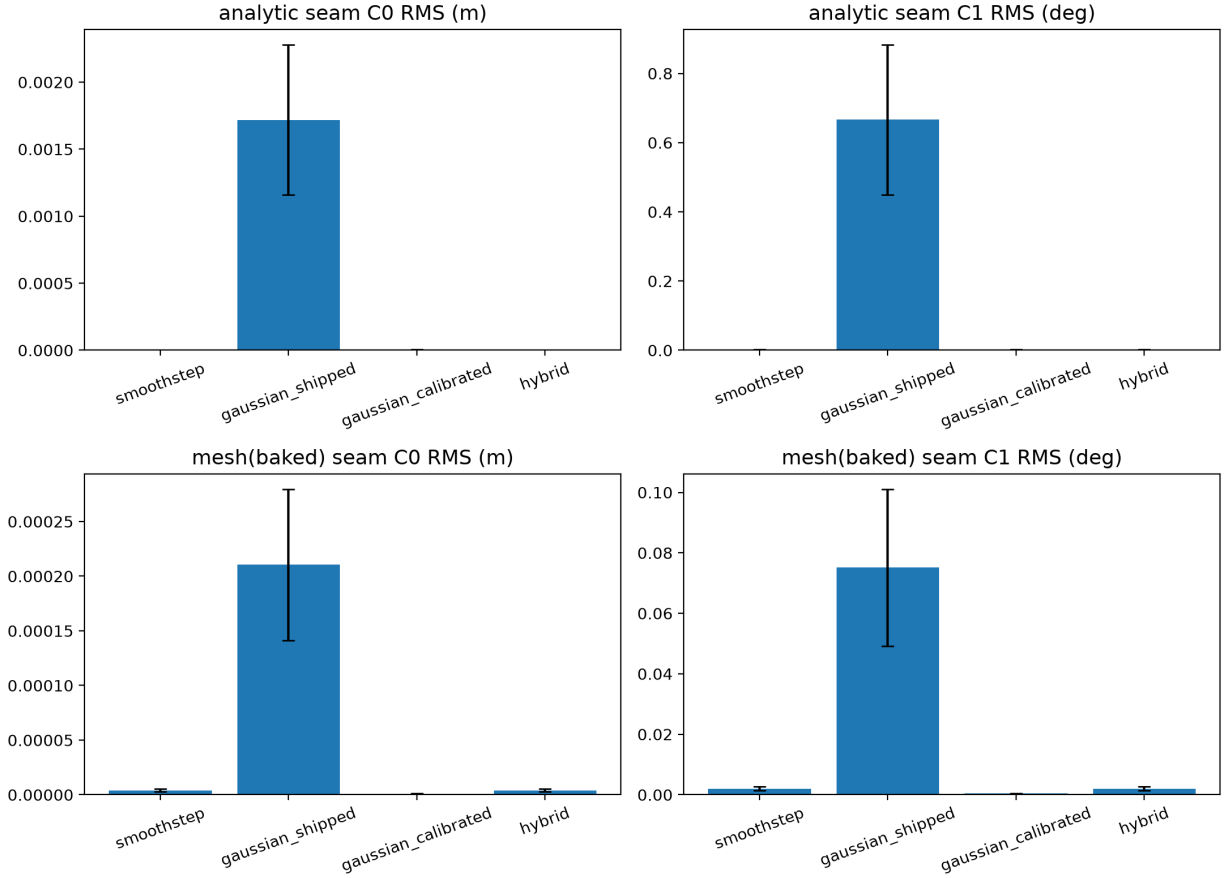


Figure 5: Cross-method comparison of C0/C1 boundary continuity.

Table 4

Estimated VRAM usage by mesh-accounting scheme.

Mesh accounting scheme	Estimated memory (MB)
Two-mesh (literal: separate global + local meshes)	5.24
Single-grid (baked: one combined mesh)	1.05

marked jump in fade-only cost and the fact that all four methods share the same VRAM footprint can be seen together in the same figure panel.

4.2. Ablation Analyses

This section presents four separate ablation analyses that test the robustness of the proposed hybrid method against four design parameters (contact radius, switch width, transition radius, grid resolution).

In the hybrid method's contact-radius (r_c) sweep (1.0–4.0 m; Fig. 7), the C0 discontinuity at the switch's own internal transition boundary ($r = r_c$) stayed below 3×10^{-4} m across all values of r_c , and the fade cost remained constant (approximately 755–793 microseconds) independent of r_c ; this confirms that the cost stems from both branches being evaluated in every case, not from where the switch is positioned.

In the switch-width (w) sweep (0.05–2.0 m; Fig. 8), the switch's internal continuity improved steadily from 7.03×10^{-4} m at $w = 0.05$ m to 1.74×10^{-7} m at $w = 1.0$ m, but degraded slightly at $w = 2.0$ m (1.83×10^{-5} m); this suggests that once the switch width approaches a magnitude comparable to the contact radius, it begins to distort the switch's local curvature, and indicates that a range of approximately

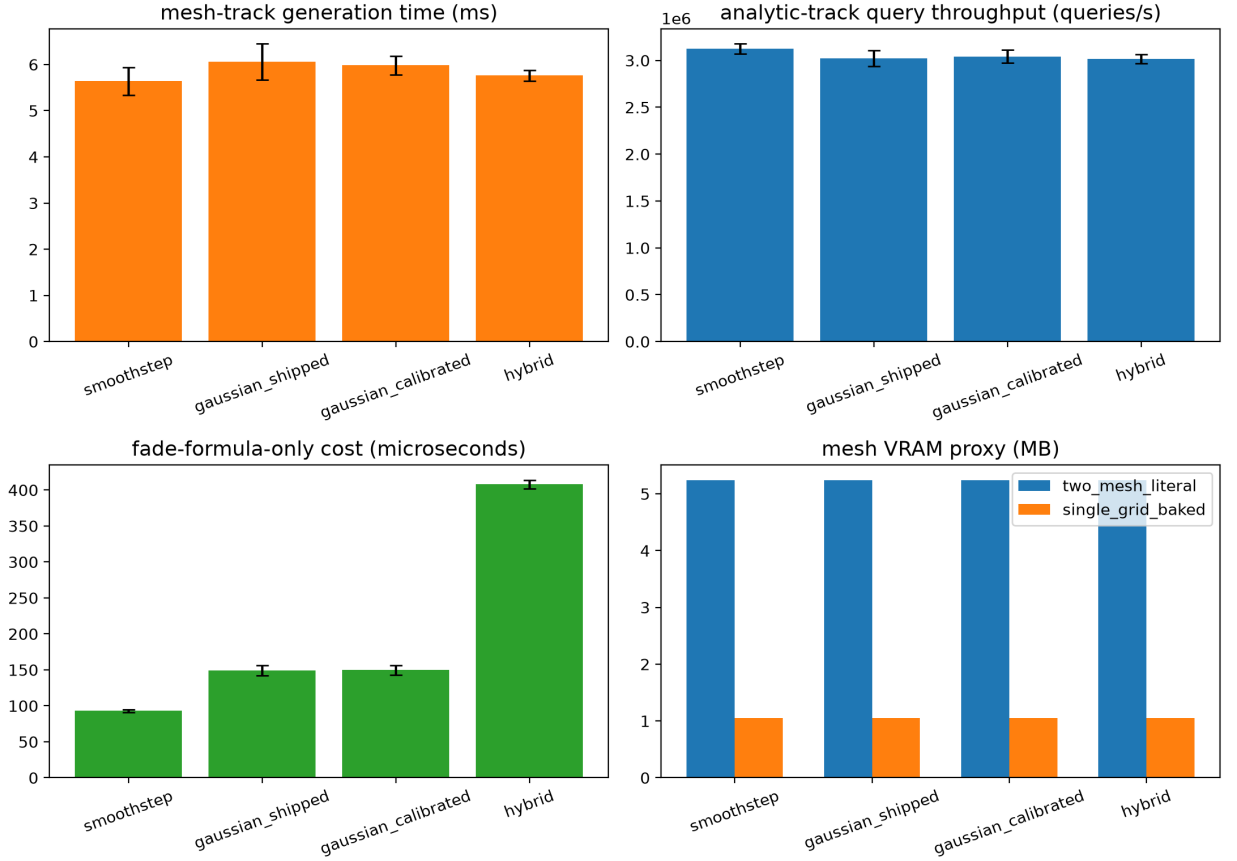


Figure 6: Cross-method comparison of computational cost.

$w = 0.3$ – 1.0 m is a suitable default for near-machine-precision internal continuity; the $w = 0.3$ m used in the production configuration provides a deliberate safety margin within this range.

In the radius sweep ($R = 4, 6, 8, 12, 16$ m; Fig. 9), carried out to test whether this defect is independent of the transition radius, the as-shipped Gaussian’s C0 (RMS) error stays nearly constant (approximately 1.68×10^{-3} to 1.76×10^{-3} m), i.e., the error does not shrink as the transition radius grows; by contrast, the error of smoothstep and the calibrated Gaussian decreases steadily as the radius grows (and curvature decreases), as expected. This shows that the as-shipped Gaussian’s defect is not a discretization error but a scale-invariant structural property independent of the transition radius.

In the grid-resolution sweep (grid spacing approximately 0.31 – 1.27 m; Fig. 10), the empirical convergence order (log-log linear-regression slope) of smoothstep and hybrid was measured at approximately 0.90 , i.e., the error decreases as expected as resolution increases. For the calibrated Gaussian, a negative slope (approximately -0.36) was observed; however, this is not a physical “gets worse as resolution increases” finding — the calibrated Gaussian’s error is already on the order of 10^{-6} to 10^{-7} m, close to a fixed truncation-residual noise floor set by the calibration constant, and this floor is a measurement limit independent of grid resolution; this point is stated explicitly here to avoid a misleading physical interpretation.

The common conclusion of the four analyses is that the proposed hybrid method’s continuity advantage is robust over a reasonable parameter range ($r_c = 1.0$ – 4.0 m, $w = 0.3$ – 1.0 m), i.e., it is not overly sensitive to a particular fine-tuning. The one exception is that at very narrow switch widths ($w \leq 0.1$ m) internal continuity visibly degrades; this once again confirms that the default used in the production configuration ($w = 0.3$ m) carries a deliberate safety margin.

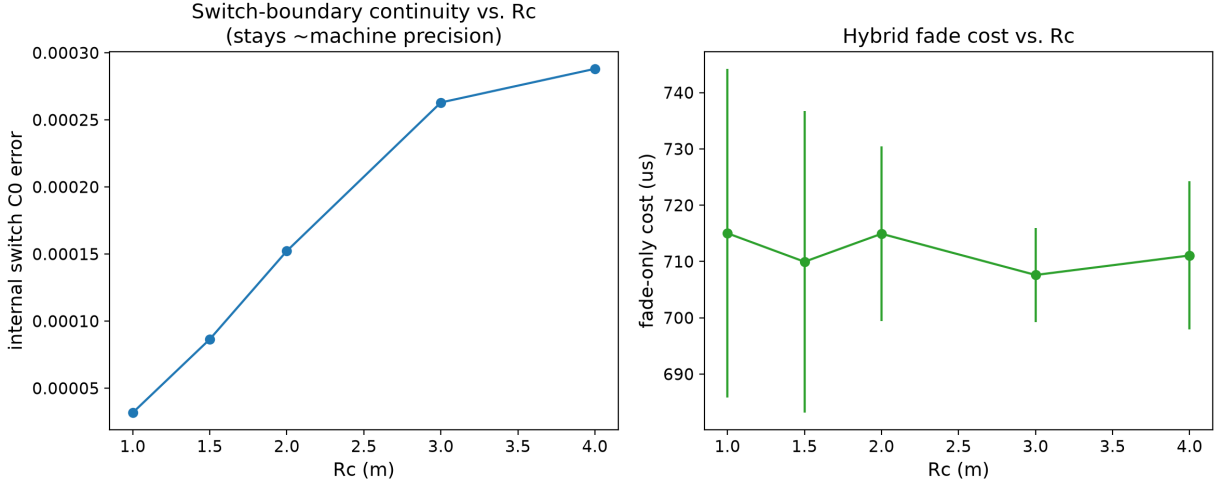


Figure 7: Contact-radius (r_c) sweep: switch internal continuity and cost.

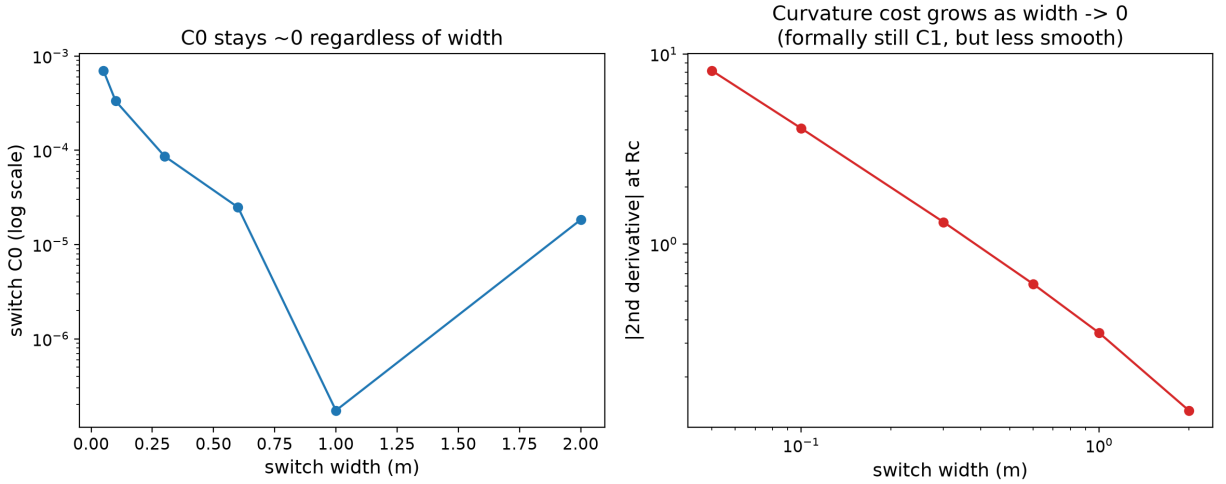


Figure 8: Switch-width (w) sweep: internal continuity.

4.3. Real Isaac Sim Training Results

All nine training runs (3 methods $\times$ 3 seeds) completed without error, each reaching 5,734,400 environment steps (Table 5). Wall-clock time, as a per-method average, was 2377 ± 98 s for smoothstep, 2850 ± 898 s for the calibrated Gaussian, and 2674 ± 599 s for the hybrid. These averages show that the large gap observed in the initial (seed=42) runs (Gaussian and hybrid respectively 56% and 35% slower) shrinks considerably with multiple seeds: the seed=7 and seed=123 runs are all close to each other (2318–2345 s), while only the seed=42 Gaussian and hybrid runs are noticeably slower. The magnitude of the standard deviations (898 and 599 s, 20–30% of the mean) is the source of this single-run deviation; the initial “35–56% slower” finding may therefore stem not from the blending formula itself but from a system-level variability specific to that particular run (background load, thermal state, or terrain-pool timing); averaged over three seeds, Gaussian and hybrid still appear slower than smoothstep (+20% and +12.5%), but this gap is no longer as sharp as in the initial measurement.

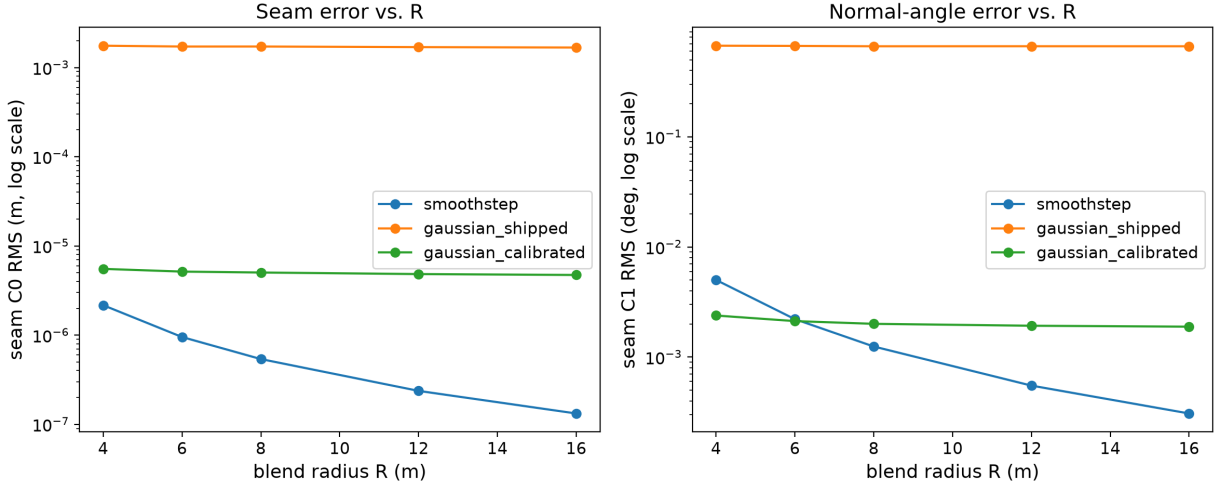
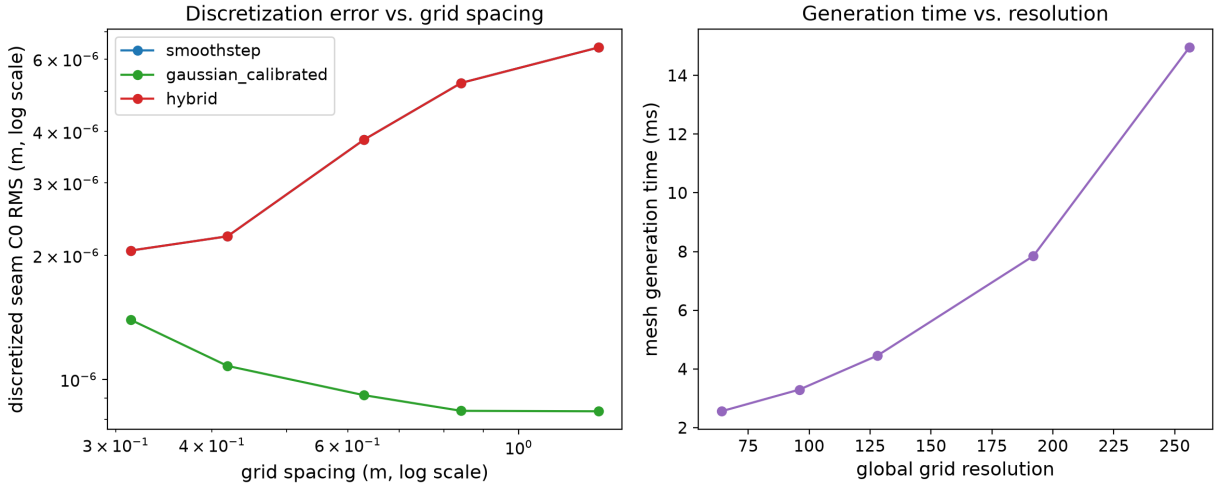

 Figure 9: Transition-radius (R) sweep: scale invariance.


Figure 10: Grid-resolution sweep: empirical convergence order.

Table 5

Summary of the final 10% window in real Isaac Sim training (three seeds per method).

Method	n	Wall time (s)	Success rate	Timeout rate	Harsh-landing
smoothstep	3	2377 ± 98	0.278 ± 0.055	0.499 ± 0.059	0.220 ± 0.022
gaussian (cal.)	3	2850 ± 898	0.325 ± 0.051	0.454 ± 0.074	0.218 ± 0.038
hybrid	3	2674 ± 599	0.268 ± 0.025	0.515 ± 0.019	0.214 ± 0.007

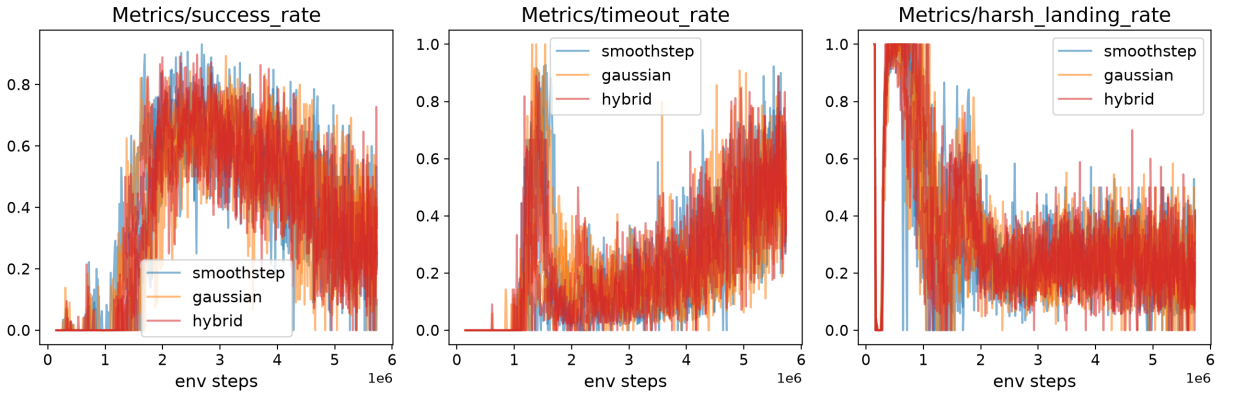
The final success rate measured in the last 10% window was, as a per-method average, $27.8\% \pm 5.5\%$ for smoothstep, $32.5\% \pm 5.1\%$ for the calibrated Gaussian, and $26.8\% \pm 2.5\%$ for the hybrid (Table 5). The Gaussian's average is still the highest, but the standard deviations of the three methods overlap with each other. Table 6 summarizes the pairwise statistical tests of these differences: no pairwise comparison is statistically significant; for success rate, smoothstep-Gaussian $p = 0.20$, smoothstep-hybrid $p = 1.00$, Gaussian-hybrid $p = 0.40$; for timeout rate, respectively $p = 0.70$, $p = 1.00$, $p = 0.40$; for harsh-landing

Table 6

Pairwise comparisons over the final 10% window (Mann-Whitney U, descriptive; see the caveat in Section 3.5).

Metric	Comparison	n	Mean diff	95% CI	MW p
Success rate	smoothstep vs gaussian	3 vs 3	-0.047	[-0.112, 0.018]	2.00e-01
Success rate	smoothstep vs hybrid	3 vs 3	0.010	[-0.040, 0.068]	1.00e+00
Success rate	gaussian vs hybrid	3 vs 3	0.057	[-0.000, 0.104]	4.00e-01
Timeout rate	smoothstep vs gaussian	3 vs 3	0.045	[-0.044, 0.133]	7.00e-01
Timeout rate	smoothstep vs hybrid	3 vs 3	-0.017	[-0.073, 0.039]	1.00e+00
Timeout rate	gaussian vs hybrid	3 vs 3	-0.061	[-0.132, 0.008]	4.00e-01
Harsh-landing	smoothstep vs gaussian	3 vs 3	0.002	[-0.041, 0.040]	1.00e+00
Harsh-landing	smoothstep vs hybrid	3 vs 3	0.006	[-0.012, 0.031]	1.00e+00
Harsh-landing	gaussian vs hybrid	3 vs 3	0.004	[-0.027, 0.047]	7.00e-01

Isaac Lab SAC training: outcome metrics (all seeds)

**Figure 11:** Outcome metrics tracked during training for the three methods (success/timeout/harsh-landing rate).

rate, all three $p \geq 0.70$. All bootstrap 95% confidence intervals include zero (e.g., for the Gaussian-hybrid success-rate difference, $[-0.000, 0.104]$ — borderline but still containing zero).

Fig. 11 shows all nine runs of the three methods for the three main outcome metrics (success, timeout, and harsh-landing rate) on the same plot; the fact that all nine curves remain largely interleaved throughout training confirms that the cross-method difference is visually indistinguishable from cross-seed variance. Fig. 12 shows complementary training diagnostics (curriculum difficulty, entropy coefficient, xy-progress reward) for the same nine runs; the entropy coefficient following an almost identical convergence curve across all three methods indicates that SAC’s exploration-exploitation balance operates independently of the choice of blending method. Fig. 13 presents the final-10% window values as per-method scatter points with a mean $\pm$ standard deviation bar; the wide overlap of the three methods’ error bars is also clearly visible here.

This result does not confirm the finding observed in the initial (single-seed) measurement — discussed in detail in Section 5 — that “Gaussian and hybrid are significantly more successful than smoothstep.” Going from $n = 1$ to $n = 3$, the effect size shrank (the Gaussian-smoothstep gap dropped from 11% to 4.7%) and the direction partly reversed (the hybrid-smoothstep gap changed sign: in the initial measurement hybrid was higher than smoothstep, while in the three-seed average hybrid is nearly equal to smoothstep, even slightly lower). This pattern indicates that the initially apparent difference is — at least largely — consistent with ordinary run-to-run (seed) variance; this point is discussed in detail in Section 5.

A similar picture holds for sample-efficiency metrics: AUC (the average success rate over training) varies over a wide range across the three seeds per method (smoothstep 0.381–0.395, Gaussian 0.366–0.402, hybrid 0.339–0.399), and the cross-method difference is markedly smaller than the cross-seed difference — again pointing to seed variance as the dominant source.

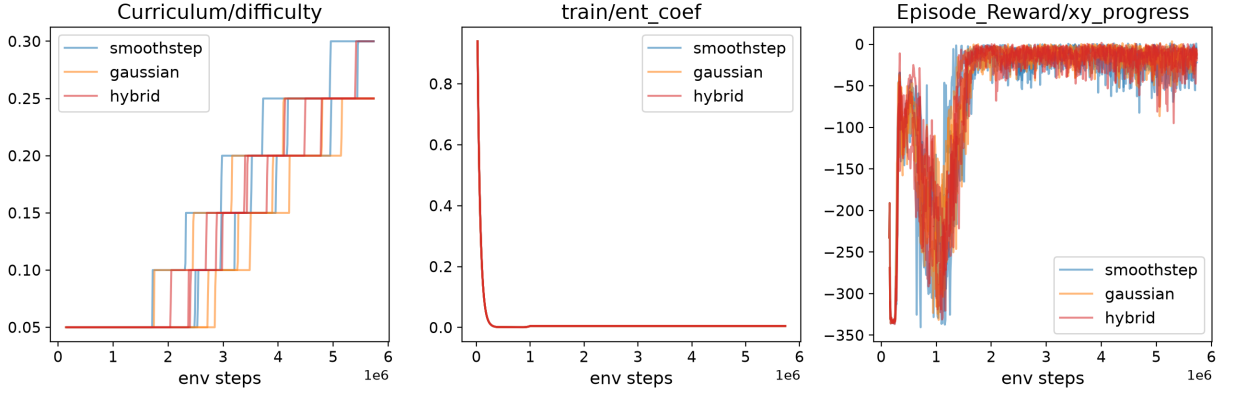


Figure 12: Complementary diagnostics tracked during training for the three methods (curriculum difficulty, entropy coefficient, xy-progress reward).

Across-seed final-window values
(independent samples -- see statistics.json caveat)

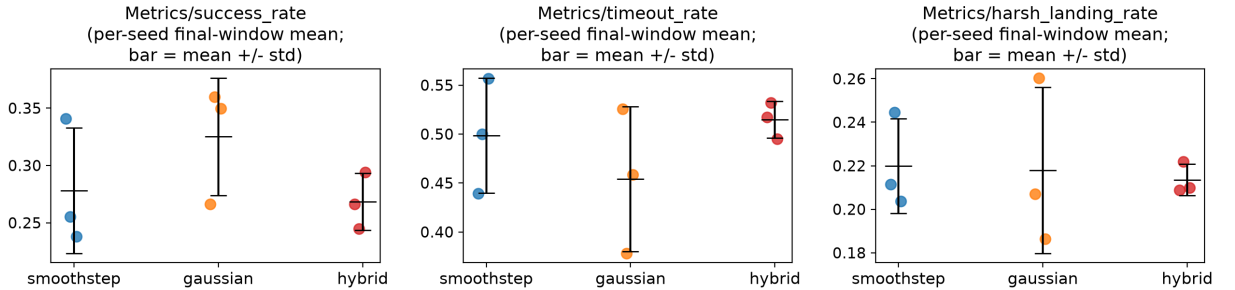


Figure 13: Distributions over the final 10% window (box plot).

Curriculum difficulty is, on average, close across all three methods (smoothstep 0.276 ± 0.025 , Gaussian 0.250 ± 0.000 , hybrid 0.260 ± 0.017); it is notable that the Gaussian stays at exactly 0.25 in all three seeds (zero standard deviation), indicating that the rolling-window success threshold sat at the same rung in all three runs — this is consistent with the observation, raised in the initial measurement, that “the Gaussian lags in curriculum despite a high average success rate,” although the cause was not directly tested.

5. Discussion

The findings first give a clear answer to RQ1: the seam left by the as-shipped Gaussian blend when truncated at a finite radius is not a cosmetic or negligible defect but a structural one, measurable at three orders of magnitude relative to smoothstep (approximately 3200x in C0, 530x in C1) and statistically highly significant. The source of this defect is not a coding error but a property of radial basis functions that has been known for more than fifty years: the Gaussian-type radial functions defined by Hardy [7] are infinite-support and never reach exactly zero at any finite radius; Wendland [8] proposed compact-support, minimal-degree positive-definite radial functions as exactly the solution to this problem. This study’s contribution is to demonstrate this fifty-year-old mathematical fact quantitatively in a concrete engineering setting (a physically

queryable lunar-terrain height field) and to test whether it is reflected in RL training. Unlike the RL results discussed below, this analytic finding is validated over 30 seeds and is not subject to single-run uncertainty; it is therefore the study’s most robust and least debatable finding.

In answer to RQ2, the proposed hybrid method preserves the Gaussian’s soft “bell” profile in the contact region and smoothstep’s compact support beyond it, while creating no new seam at its own internal transition (switch); this is an expected result, consistent with the partition-of-unity framework described by Ohtake et al. [10]: a convex combination of two C1-smooth functions with a C1-smooth weight is itself C1-smooth. The cost of this advantage is an approximately 4.3x higher microsecond cost than smoothstep in computing the pure fade formula (because both branches must be evaluated). This formula-level cost is reflected only partially and inconsistently in the real training’s wall-clock time: in the initial (seed=42) measurement, Gaussian and hybrid were 56% and 35% slower, respectively, but averaged over three seeds this gap drops to 20% and 12.5%, and the standard deviation (20–30% of the mean) shows how much a single run can deviate (Section 4.3). This suggests that the real wall-clock difference stems largely from run-to-run system variability (background load, thermal state, terrain pool timing), with the pure formula cost contributing only a small and consistent part; this is a separate observation, not to be over-interpreted, but one that shows single-run measurements can also be misleading for cost comparisons.

The answer to RQ3 changed substantially after multiple seeds, and this change is itself one of the study’s most instructive findings. In the initial (single-seed, seed=42) measurement, Gaussian and hybrid appeared to reach a final success rate approximately 8–12 percentage points higher than smoothstep, and this initial draft attributed it to a mechanism such as “a difference in the transition-band weight profile.” In the measurement extended with two additional independent seeds (seed=7, seed=123) (Section 4.3), the cross-method gap shrank (the Gaussian-smoothstep gap dropped from 11% to 4.7%), the hybrid-smoothstep gap changed sign, and no pairwise comparison was statistically significant on any of the three metrics (success rate, timeout rate, harsh-landing rate; all $p \geq 0.20$). Recalling that the Gaussian variant used in the real training (calibrated, $k = \ln(1000)$) already has analytic boundary continuity nearly equal to smoothstep’s (Table 2), this result is in fact an expected consistency: if there is no measurable difference in boundary continuity, it is not surprising that there is also no strong difference in RL training performance. The apparent gap in the initial measurement was most likely ordinary run-to-run variance stemming from the GPU-parallel PhysX simulation’s own internal nondeterminism and the timing-dependence of the terrain pool’s background CPU generation process — exactly the pseudoreplication pitfall warned against by Henderson et al. [16]. Because this study thereby refutes its own initial finding, it offers a self-contained case study on how misleading single-run RL comparisons can be: going from $n = 1$ to $n = 3$ was enough to render an apparently “significant” finding statistically indistinguishable. That said, $n = 3$ is still low-powered, and the present data cannot rule out the presence of a small real effect (especially given that the Gaussian consistently has the highest average); a larger number of seeds is needed to reach a definitive conclusion (Section 6).

The scope of this study was deliberately kept narrow in order to isolate the effect of the blending function as a single variable; the three methods were compared only against each other, with the rest of the production environment (physics engine, reward function, agent architecture, curriculum schedule) held fixed. Bringing in another terrain-blending method from the literature — e.g., one implemented in a different simulator — as an external benchmark was deliberately not chosen; such a comparison would make it impossible to separate the effect of the blending function from the countless other differences in that simulator’s physics engine, observation/reward design, and agent architecture. The cost of this choice is that the extent to which the findings generalize beyond this specific environment (Isaac Lab/PhysX, SAC, this task definition) — external validity — was not tested in this study; this point is addressed as a concrete future-work direction in Section 6.

The most important limitation of this study is that, with $n = 3$, it still has low statistical power: the smallest p -value the Mann-Whitney test can produce is already bounded with the current data (around a minimum of $p \approx 0.10$ for a 3-vs-3 comparison), so an “no significant difference” finding does not prove that the methods are truly equivalent — it only shows that the seemingly large gap in the initial measurement could not be reliably confirmed. The second limitation is that the training scale (5.73 million steps, approximately 350 iterations) remained below the full-scale configuration (1200 iterations) owing to the computational budget; no run may have reached full convergence, meaning the observed plateaus may reflect an interim-stage performance difference rather than a final/asymptotic one. The third limitation is that the analytic/CPU

study isolates only the fade formula itself and does not fully disentangle the source of the real-training wall-clock difference (terrain generation, physics stepping, system variability) — the three-seed data show that this gap shrinks, but do not conclusively identify its source.

6. Conclusion

This study systematically examined the shape of the blending function used where a coarse-resolution global terrain meets a fine-detail local patch in an Isaac Lab-based lunar-landing environment, both at the analytic/CPU level and in a real Isaac Sim SAC training run. Analytic measurements over thirty seeds statistically confirmed that the previously used as-shipped Gaussian blend produces, once truncated at a finite radius, boundary continuity worse than smoothstep by three orders of magnitude, and that this defect is a scale-invariant, structural property. The proposed hybrid method and a calibrated Gaussian variant remove this defect to near-machine precision — this is the study’s most robust and least debatable finding.

The real Isaac Sim training comparison began with a single seed per method, and the initial measurement showed Gaussian and hybrid reaching a markedly higher success rate than smoothstep. To test the reliability of this finding, the study was extended with two additional independent seeds (total $n = 3$), and no statistically significant difference was found between the methods on any metric. This is the study’s second major contribution: it provides a concrete, self-contained example of how misleading single-run RL comparisons can be, and empirically confirms the warning of Henderson et al. [16] in this specific setting. Taken together with the analytic results, this shows that the choice of blending function has a measurable and robust effect on boundary continuity, but that this effect — at least at the present scale and with $n = 3$ — is not reflected in final RL training performance in a distinguishable way.

The core contribution of this study is a methodology that tests the choice of blending function quantitatively, both analytically and at the level of RL training, in a real Isaac Sim/Isaac Lab environment, rather than treating it as a purely visual/rendering preference; this methodology is able to reliably report a “no significant difference” result just as well as a positive one. All three methods are integrated into the production code as a backward-compatible, selectable configuration, making these findings directly actionable: with the current data, there is no statistical justification for choosing among the three methods on RL training performance; the choice can instead be based on other criteria such as boundary continuity (analytically in favor of Gaussian and hybrid) or computational cost (in favor of smoothstep).

Three concrete directions are proposed for future work. First, because $n = 3$ still has low statistical power, the real-training comparison should be extended with at least five to ten more independent seeds to determine conclusively whether the small observed average differences (particularly the Gaussian’s consistently highest average) reflect a real effect or residual noise. Second, the training scale should be extended to the full configuration (1200 iterations) to confirm whether the observed plateaus reflect an interim-stage rather than a final/asymptotic performance difference. Third, going beyond the scope that this study deliberately kept narrow (see Section 5), the external validity of the findings should be tested: repeating the same three blending methods on a different physics engine, a different reward/observation design, or a different contact-rich task (e.g., quadrupedal locomotion, humanoid standing) would disentangle whether the findings here depend on the blending function or on other design choices specific to this particular environment.

Acknowledgment

This work was carried out without any external funding or institutional support, using the author’s own computational resources (a local NVIDIA GPU workstation). The author thanks the NVIDIA team for developing Isaac Lab and Isaac Sim as open-source software, and the open-source software libraries used in the experimental infrastructure (Stable-Baselines3, PyTorch, NumPy, SciPy, TensorBoard).

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships related to this work.

Data and code availability

The analytic evaluation code used in this study (the NumPy port of the three blending functions, the continuity/cost metrics, and the ablation scripts), the production environment code (the Isaac Lab task definition and the selectable integration of the three methods via `ISAACLAB_TERRAIN_BLEND_METHOD`), and the raw experimental outputs (the thirty-seed continuity measurements, the four ablation sweeps, and the real-training TensorBoard logs) are stored in the author's project repository and are available from the author upon reasonable request. The code is planned to be moved to a publicly accessible repository during the manuscript review process.

CRedit authorship contribution statement

Haktan Yusuf Güloğlu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data curation, Writing – original draft, Writing – review & editing, Visualization, Supervision, Project administration.

References

- [1] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, Orbit: A unified simulation framework for interactive robot learning environments, *IEEE Robotics and Automation Letters* 8 (6) (2023) 3740–3747. doi:10.1109/LRA.2023.3270034.
- [2] NVIDIA, Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning (2025). arXiv:2511.04831. URL <https://arxiv.org/abs/2511.04831>
- [3] N. Rudin, D. Hoeller, P. Reist, M. Hutter, Learning to walk in minutes using massively parallel deep reinforcement learning, in: *Proceedings of the 5th Conference on Robot Learning (CoRL 2022)*, Vol. 164 of PMLR, 2022, pp. 91–100. URL <https://proceedings.mlr.press/v164/rudin22a.html>
- [4] F. Losasso, H. Hoppe, Geometry clipmaps: Terrain rendering using nested regular grids, *ACM Transactions on Graphics* 23 (3) (2004) 769–776. doi:10.1145/1015706.1015799.
- [5] M. Duchaineau, M. Wolinsky, D. E. Sigi, M. C. Miller, C. Aldrich, M. B. Mineev-Weinstein, ROAMing terrain: Real-time optimally adapting meshes, in: *Proceedings of the 8th IEEE Conference on Visualization (VIS '97)*, 1997, pp. 81–88. URL <https://ieeexplore.ieee.org/document/663860/>
- [6] W. H. de Boer, Fast terrain rendering using geometrical mipmapping, Tech. rep., E-mersion Project (2000). URL https://www.flipcode.com/archives/article_geomipmaps.pdf
- [7] R. L. Hardy, Multiquadric equations of topography and other irregular surfaces, *Journal of Geophysical Research* 76 (8) (1971) 1905–1915. doi:10.1029/JB076i008p01905.
- [8] H. Wendland, Piecewise polynomial, positive definite and compactly supported radial basis functions of minimal degree, *Advances in Computational Mathematics* 4 (1) (1995) 389–396. doi:10.1007/BF02123482.
- [9] D. S. Ebert, F. K. Musgrave, D. Peachey, K. Perlin, S. Worley, W. R. Mark, J. C. Hart, *Texturing and Modeling: A Procedural Approach*, 3rd Edition, Morgan Kaufmann, 2002.
- [10] Y. Ohtake, A. Belyaev, H.-P. Seidel, A multi-scale approach to 3D scattered data interpolation with compactly supported basis functions, in: *Proceedings of the 2003 Shape Modeling International (SMI '03)*, 2003, pp. 153–161. URL <https://ieeexplore.ieee.org/document/1199611/>
- [11] A. E. Johnson, J. F. Montgomery, Overview of terrain relative navigation approaches for precise lunar landing, in: *2008 IEEE Aerospace Conference*, 2008, pp. 1–10. doi:10.1109/AERO.2008.4526302.
- [12] NASA Jet Propulsion Laboratory, ALHAT: Autonomous landing and hazard avoidance technology, JPL Robotics (n.d.). URL <https://www-robotics.jpl.nasa.gov/what-we-do/research-tasks/alhat-autonomous-landing-and-hazard-avoidance-tech>
- [13] M. K. Barker, E. Mazarico, G. A. Neumann, M. T. Zuber, J. Haruyama, D. E. Smith, A new lunar digital elevation model from the Lunar Orbiter Laser Altimeter and SELENE Terrain Camera, *Icarus* 273 (2016) 346–355. doi:10.1016/j.icarus.2015.07.039.
- [14] T. Ishida, S. Fukuda, K. Kariya, H. Kamata, K. Takadama, H. Kojima, S. Sawai, S.-i. Sakai, Vision-based navigation and obstacle detection flight results in SLIM lunar landing, *Acta Astronautica* 226 (2025) 772–781. doi:10.1016/j.actaastro.2024.11.002.
- [15] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. doi:10.1109/IROS.2017.8202133.
- [16] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, *Proceedings of the AAAI Conference on Artificial Intelligence* 32 (1) (2018) 3207–3214. doi:10.1609/aaai.v32i1.11694.



Haktan Yusuf Güloğlu graduated at the top of his class from Profilo Vocational and Technical Anatolian High School and is currently an undergraduate student in Computer Engineering at Sakarya University. He

works as a student researcher at Sakarya University's National Technology Workshop Artificial Intelligence Center, where his research interests center on reinforcement learning, procedural simulation environments, and the numerical accuracy of contact geometry in physics engines. He carried out this study's entire pipeline — from conceptualization and experimental design through the software implementation to the statistical analysis and writing — as its sole author.